

ЛР1. Реалізація та перевірка зворотного поширення похибки

Дедлайн: до початку лекції 6. **Оцінювання:** 10 балів. **Стек:** Python 3.12, NumPy, PyTorch.
Обчислення у `float64`.

Мета роботи

Реалізувати зворотне поширення похибки (backpropagation) вручну та перевірити отримані градієнти двома способами: за допомогою PyTorch autograd і чисельного диференціювання.

Після виконання роботи ви маєте вміти пояснити, звідки береться кожен градієнт, які проміжні значення потрібні для його обчислення та як виявити помилку у зворотному проході. Основні перевірки виконуються **до навчання**; мінімальна точність класифікації не встановлюється.

Вихідні дані та фіксовані налаштування

Завантажте Iris через `sklearn.datasets.load_iris()`: 150 об'єктів, 4 числові ознаки, 3 класи.

Стратифікований поділ 70/30: по 35 об'єктів кожного класу в навчальній вибірці та по 15 у тестовій. Для відтворюваності використайте `np.random.default_rng(0)`: переберіть класи в порядку 0, 1, 2, перемішайте індекси кожного класу та візьміть перші 35 для навчання.

Стандартизуйте кожну ознаку: відніміть середнє та поділіть на стандартне відхилення. Обидві статистики обчислюйте лише на навчальній вибірці; для стандартного відхилення використайте `ddof=0`. До тестової вибірки застосуйте ті самі статистики.

Мережа:

$$X \rightarrow \text{Linear}(4, 8) \rightarrow \text{ReLU} \rightarrow \text{Linear}(8, 3).$$

Параметр	Розмірність	Ініціалізація
W_1	4×8	He: нормальний розподіл із середнім 0 та стандартним відхиленням $\sqrt{2/4}$
b_1	8	Нулі
W_2	8×3	Xavier: нормальний розподіл із середнім 0 та стандартним відхиленням $\sqrt{2/(8 + 3)}$
b_2	3	Нулі

Для ініціалізації створіть новий `np.random.default_rng(0)` і послідовно згенеруйте W_1 , потім W_2 . Архітектуру й початкові налаштування не змінюйте.

Вихід мережі - логіти трьох класів. Функція втрат - крос-ентропія, **усереднена за об'єктами**. Для всіх основних перевірок використайте повну навчальну вибірку з 105 об'єктів і початкові параметри.

1. Реалізація в NumPy

Реалізуйте:

- прямий прохід двох афінних шарів із ReLU;
- чисельно стабільну крос-ентропію через log-softmax зі зсувом на максимум;
- збереження проміжних значень для зворотного проходу;
- ручне обчислення градієнтів за W_1, b_1, W_2, b_2 .

У NumPy-реалізації не використовуйте автоматичне диференціювання. PyTorch застосовуйте лише в окремій еталонній реалізації для звірки.

У звіті (README, довільного розширення – docx/pdf/md) запишіть формули зворотного проходу та розмірності проміжних масивів і чотирьох градієнтів.

2. Звірка з PyTorch

Створіть еквівалентну мережу в PyTorch та **скопійуйте в неї параметри NumPy-мережі**. Врахуйте, що `torch.nn.Linear` зберігає ваги як `(out, in)`.

Порівняйте втрату та всі елементи чотирьох градієнтів на тих самих даних. В README подайте таблицю:

Величина	Максимальна абсолютна різниця NumPy / PyTorch	Перевірку пройдено
Втрата		
Градiєнт W_1		
Градiєнт b_1		
Градiєнт W_2		
Градiєнт b_2		

Окремо наведіть самі значення втрати NumPy та PyTorch. Для кожної пари скалярних значень a, b використайте критерій:

$$|a - b| \leq 10^{-12}$$

Усі порівнювані значення мають бути скінченними. Якщо перевірка не проходить – знайдіть і поясніть причину.

3. Перевірка окремих градієнтів чисельним диференціюванням

Перевірте вручну реалізовані градієнти для чотирьох параметрів:

$$W_1[0, 0], \quad b_1[0], \quad W_2[0, 0], \quad b_2[0].$$

Використайте початкові ваги та повну навчальну вибірку, як у пункті 2.

Для кожного вибраного параметра θ_i :

1. Збережіть його початкове значення.

- Збільште **лише цей параметр** на $\varepsilon = 10^{-6}$ та обчисліть втрату L_+ .
- Установіть значення на ε менше від початкового та обчисліть L_- .
- Відновіть початкове значення й обчисліть чисельну похідну:

$$g_{\text{num},i} = \frac{L_+ - L_-}{2\varepsilon}.$$

Порівняйте її з відповідним елементом градієнта, який повертає ваш `backward()` на початкових вагах.

Подайте таблицю:

Параметр	Градієнт <code>backward()</code>	Чисельна похідна	Абсолютна різниця	Перевірку пройдено
$W_1[0, 0]$				
$b_1[0]$				
$W_2[0, 0]$				
$b_2[0]$				

Критерій проходження для кожного параметра:

$$|g_{\text{num}} - g_{\text{manual}}| \leq 10^{-7}$$

4. Дослід із навмисною помилкою

Тимчасово приберіть ділення на кількість об'єктів у градієнті за логітами, залишивши функцію втрат незмінною. Використайте ті самі початкові параметри і ті самі 105 об'єктів; не виконуйте оновлення ваг між порівняннями.

Перед запуском запишіть прогноз:

- чи зміниться втрата на тих самих параметрах;
- як зміняться ненульові градієнти;
- які перевірки мають виявити помилку.

Повторіть звірку з PyTorch та чисельну перевірку. Еталон PyTorch і формулу чисельної похідної не змінюйте. Зіставте результат із прогнозом. Після дослідів відновіть правильну реалізацію.

Правильний результат цього дослідів - виявлена помилка.

Збережіть спосіб відтворення обох реалізацій, наприклад окремою функцією або параметром запуску. За замовчуванням має працювати правильна реалізація.

Що здати

Теку `lab1/` у (індивідуальному для кожного студента) спільному github репозиторії всіх лабораторних робіт, яка містить:

- код у **.py**-файлах, файл залежностей із зафіксованими версіями та команду відтворення через **uv**;
- результати звірки з PyTorch;
- таблицю чотирьох чисельних перевірок;
- прогноз і результати дослід з навмисною помилкою та команду його відтворення;
- **README** із формулами, розмірностями, поясненням збережених проміжних значень і висновками.

Оцінювання та захист

Складова	Бали
Прямий прохід і стабільна функція втрат	2
Ручний backprop, формули та розмірності	3
Звірка з PyTorch і чисельна перевірка	2
Дослід із помилкою та пояснення результатів	2
Відтворюваність і повнота звіту	1
Разом	10

На захисті студент пояснює власні формули, код і результати перевірок. Оцінюється виконання та розуміння кожної складової.

Використання зовнішнього коду та генеративних інструментів дозволене із зазначенням джерел у **README**; воно не скасовує вимоги розуміти поданий розв'язок. Для повного зарахування результат має відтворюватися за наведеною командою на машині викладача.